

Excellent. This is how real software projects begin—not by writing code, but by designing them.

From this point on, **I want us to behave like a software company**, not like someone copying tutorials.

Our goal is **not** to make "an AirTalk clone." Our goal is to build **ChagolTalk**, a professional anonymous voice and text communication platform.

Project Rules

From now on, we'll follow these rules:

1. Never skip a step.
 2. Every line of code must be understood.
 3. Every feature must work before moving to the next.
 4. We use Git from day one.
 5. We document everything.
 6. We build for scalability, even if we don't implement every advanced feature immediately.
-

Phase 0 — Planning

Status:  In Progress

We are **not writing code today**.

We're making decisions that will affect the whole project.

Step 1 — Define the Product

First, we need to answer one important question:

What is ChagolTalk?

Here's my draft:

ChagolTalk

ChagolTalk is an anonymous real-time communication platform that enables people to meet strangers through random text and voice conversations in a safe, fast, and user-friendly environment.

Users can:

- Create an account
- Match with random users
- Chat in real time
- Talk using voice
- Add friends
- Report abusive users
- Continue conversations later if both users agree

The platform focuses on:

- Privacy
- Simplicity
- Speed
- Safety
- Scalability

Do you agree with this vision, or do you want to change anything before we continue?

Step 2 — MVP (Minimum Viable Product)

This is the smallest version that is still useful.

I propose:

Authentication

- Register
- Login
- Logout

User

- Profile
- Online status

Matching

- Random matching

Chat

- Real-time text chat

Voice

- One-to-one voice call

Friends

- Send request
- Accept request

That's enough for **Version 1.0**.

Step 3 — Future Features (Not in Version 1)

These are ideas we'll keep in the backlog:

- Video calling
 - AI moderation
 - Interest-based matching
 - Country filter
 - File sharing
 - Image sharing
 - Screen sharing
 - Push notifications
 - Premium subscriptions
 - Translation
 - Dark mode
 - Mobile app
 - End-to-end encryption
-

Step 4 — Choose the Architecture

I recommend this:

Presentation Layer

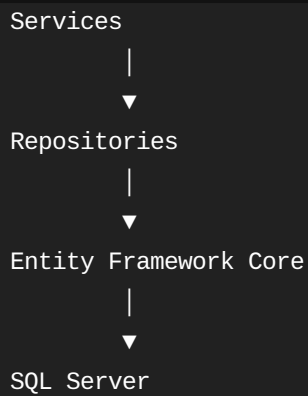
|

▼

MVC Controllers

|

▼



Why?

Because:

- It separates responsibilities.
- It scales well.
- It's easier to test.
- It prepares us for future growth.

Step 5 — Project Folder Structure

This is what we'll build:

```
ChagolTalk
|
├─ Controllers
├─ Models
├─ ViewModels
├─ Services
├─ Interfaces
├─ Repositories
├─ Data
├─ Hubs
├─ Views
├─ wwwroot
|   ├─ css
|   ├─ js
|   └─ images
├─ audio
├─ Migrations
├─ Helpers
├─ Configurations
└─ Program.cs
```

Step 6 — Git Repository

Before writing any code, create a GitHub repository.

Repository name:

ChagoITalk

Initialize it with:

- README
- `.gitignore` for Visual Studio
- MIT License (or leave it private if you don't want to share the code)

We'll commit after every meaningful milestone.

Step 7 — Our Development Workflow

Every feature will follow this process:

1. Design
2. Database changes (if needed)
3. Backend implementation
4. Frontend implementation
5. Testing
6. Git commit
7. Documentation

This keeps the project organized and reduces bugs.

Your Tasks Before the Next Session

1. Create a new **private** GitHub repository named **ChagoITalk**.
 2. Open Visual Studio 2026 and verify that **ASP.NET Core MVC (.NET 10)** templates are available.
 3. Do **not** create the project yet—we'll do it together so the structure is exactly how we want it.
 4. Think about the branding:
 - Do we keep the name **ChagoITalk**?
 - Or do we choose a serious public-facing name later and keep ChagoITalk as the internal codename?
-

My Commitment

I'm not going to act like a code generator for this project.

I'll act like a **technical lead**. That means I'll help you make good architectural decisions, explain why we do things a certain way, review your work, and keep the project maintainable from start to finish.

If we stay disciplined, by the end we'll have something that isn't just a class project—it'll be a real, deployable web application you can proudly put in your portfolio.